

Project 4

Accelerator-Aware GEMM using gem5 RISC-V

Weight: 10% of course grade

Submission format: One ZIP file uploaded to the course submission system

Submission format & Deadline: 12:00 on 06/03.

Individual work rule

This project must be completed individually. You may discuss high-level course concepts with classmates, but your code, experiments, measurements, analysis, and report must be entirely your own work. The use of ChatGPT or any other large language model (LLM), AI writing assistant, AI coding assistant, or automated paraphrasing/generation tool is strictly prohibited for this assignment. This includes, but is not limited to:

- generating code,
- generating explanations,
- generating report text,
- generating answers to analysis questions,
- rewriting your draft,
- interpreting gem5 results for you.
- Any use of such tools in preparing or submitting this assignment will be treated as academic misconduct.

Purpose of this project

In Projects 1, 2, and 3, you studied GEMM from the viewpoints of ISA-aware reasoning, pipeline-aware software organization, and memory-system-aware optimization. In this final project, you will extend the same workload into an accelerator-oriented architectural study.

The common workload for all semester projects remains dense integer matrix multiplication:

$$C = A \times B$$

This project focuses on accelerator integration rather than only CPU-side loop optimization. You will evaluate when a GEMM tile should remain on the CPU and when it should be offloaded to an accelerator-like timing abstraction.

This project is not about building an RTL accelerator or modifying gem5's RISC-V decoder.

Instead, it is about accelerator-aware architecture, offload granularity, batching, data movement overhead, throughput, and system-level tradeoffs.

You will:

- Run CPU-only GEMM baselines
- Use a provided accelerator offload interface
- Implement several offload strategies
- Measure behavior in gem5
- Analyze offload overhead, batching, granularity, utilization, and estimated energy
- Explain when accelerator offload helps and when it fails

Execution models used in this project

This project uses two gem5 CPU models:

- TimingSimpleCPU: a timing-aware simplified model
- MinorCPU: a structured in-order pipeline model

These two models should not be interpreted as exact implementations of textbook processors. They are used to provide two different execution contexts for studying the CPU-side cost of offload management, polling, setup code, and residual computation.

The accelerator used in this project is a software-visible timing abstraction. It is not a real gem5 hardware device. It models the idea that a GEMM tile is submitted to an accelerator, pays a setup cost, pays a data-movement cost, pays a modeled compute cost, and then produces the correct tile result.

Learning objectives

By the end of this project, you should be able to:

- Explain why accelerator speedup depends on offload granularity
- Distinguish compute acceleration from data-movement overhead
- Analyze batching and setup overhead amortization
- Relate cache behavior and memory hierarchy effects to accelerator offload
- Compare CPU-only and accelerator-assisted execution using gem5 statistics
- Estimate normalized energy from measured work counts and modeled costs
- Make an architecture-level recommendation using quantitative evidence

Required computing environment

All experiments for this project must be run using the following Docker image:

amansinhaatnycu/ca:latest

This Docker image already contains:

gem5 v25.1.0.0

Pull: `docker pull amansinhaatnycu/ca:latest`

Create Mount Directory: `mkdir -p ca`

Run: `docker run --rm -it -v .\ca\project4_starter:/work -w /work amansinhaatnycu/ca:latest bash`

Common workload

The common workload is dense integer matrix multiplication:

$$C = A \times B$$

You must use the provided starter program as your starting point.

The matrices are initialized in a deterministic structured form so that accelerator-mode tiles can be verified without performing all MAC operations on the CPU. You must not change the initialization rule or the correctness rule.

Files provided to students

You will be given the following starter files:

Makefile

gemm_accel.cpp

accel_model.h

run_timing.py

run_minor.py

run_all_template.sh

extract_stats.py

You may modify `gemm_accel.cpp` and the run scripts as needed, but your submission must remain organized and reproducible.

Project constraints

You must do

- Use the provided GEMM program as the starting point
- Use the Docker image `amansinhaatnycu/ca:latest`
- Use `gem5 v25.1.0.0`
- Run all required experiments
- Verify correctness for every mode
- Explain your accelerator timing assumptions clearly
- Distinguish measured `gem5` results from modeled accelerator behavior
- Include exact commands and extracted statistics

You may do

- Change tile sizes where explicitly allowed
- Implement batching and hybrid CPU-accelerator strategies
- Add helper functions for cleaner code
- Add additional modes for optional analysis
- Add additional plots or tables in your report

You must not do

- Use external math libraries such as BLAS
- Use SIMD intrinsics
- Use assembly language
- Use GPU code
- Use multithreading
- Change the mathematical problem
- Change matrix initialization or correctness checking
- Claim that you implemented a real `gem5` accelerator device
- Claim that you added a real RISC-V instruction unless you actually modified `gem5` correctly
- Skip correctness checking
- Fabricate or estimate `gem5` results without running them

Baseline build instructions

Inside the Docker container, build the program using:

`make`

Required program modes

The provided program supports the following modes:

- `baseline`
- `tiled_cpu`
- `accel_tiny`
- `accel_medium`
- `accel_large`
- `hybrid`

The baseline mode performs ordinary CPU GEMM.

The `tiled_cpu` mode performs CPU GEMM using tiles, but without accelerator offload.

The `accel_tiny`, `accel_medium`, and `accel_large` modes use the accelerator offload abstraction with different tile granularities.

The hybrid mode must combine CPU execution and accelerator offload in a way that you justify architecturally.

Required gem5 CPU models

You must run your experiments using both of the following gem5 CPU models:

- `TimingSimpleCPU`
- `MinorCPU`
- In the provided scripts, these correspond to:
 - `run_timing.py`
 - `run_minor.py`

Required matrix sizes

You must run experiments for all of the following matrix sizes:

- `N=64`
- `N=128`

- N=256

You may run additional sizes if you wish, but the three sizes above are mandatory.

Required modes for the main study

For each required matrix size, you must run:

- baseline
- tiled_cpu
- accel_tiny
- accel_medium
- accel_large

This gives a minimum of:

3 sizes x 2 CPU models x 5 modes = 30 required gem5 runs

The hybrid mode is required for Task 5 and must be run for at least N=128 and N=256 on both CPU models.

Project tasks

Task 1 - Run CPU-only baselines

Run baseline and tiled_cpu on both CPU models for all required matrix sizes.

For every run, record:

- matrix size
- CPU model
- mode
- checksum
- correctness status
- simTicks
- simInsts
- At least two relevant memory or cache statistics

Task 2 - Understand the accelerator timing abstraction

Study accel_model.h and explain the following in your report:

- setup cost per offload

- data movement cost per offload
- modeled compute cost
- tile size
- number of accelerator calls
- why very small tiles may lose performance even if the accelerator compute is fast

Task 3 - Implement accelerator offload modes

Implement or complete the following modes:

- accel_tiny: small-tile offload with high offload count
- accel_medium: medium-tile offload with moderate offload count
- accel_large: large-tile offload with low offload count

For each mode, report:

- tile size
- number of offload calls
- modeled MAC count
- modeled transfer bytes
- checksum
- correctness status

Task 4 - Granularity and batching analysis

Compare accel_tiny, accel_medium, and accel_large.

Your analysis must answer:

- Which mode has the highest offload overhead?
- Which mode best amortizes setup cost?
- Which mode gives the best measured gem5 behavior?
- Does the best tile size remain the same for $N=64$, $N=128$, and $N=256$?
- Why can a mode with fewer accelerator calls still fail to be best?

Task 5 - Hybrid CPU-accelerator execution

Implement one hybrid strategy. Examples include:

- accelerator handles full interior tiles while CPU handles boundary tiles
- accelerator handles large regular tiles while CPU handles leftover rows or columns
- CPU handles small matrices while accelerator handles only larger matrices

You must define your hybrid rule clearly.

Run hybrid for at least:

- N=128 on TimingSimpleCPU
- N=128 on MinorCPU
- N=256 on TimingSimpleCPU
- N=256 on MinorCPU

Task 6 - Energy-oriented reasoning

Use the following normalized energy model:

- CPU MAC energy = 1.00 unit
- Accelerator MAC energy = 0.30 unit
- Offload setup energy = 2000 units per accelerator call
- Data movement energy = 0.02 units per byte moved
- CPU residual loop energy = 0.20 units per executed instruction

Using your measured and modeled quantities, estimate normalized energy for at least:

- baseline
- accel_medium
- hybrid

You must show your formulas and intermediate quantities.

Task 7 - Final architecture recommendation

Make one final recommendation:

- CPU-only execution
- accelerator-only offload
- hybrid CPU-accelerator execution

Your recommendation must be based on measured results and modeled energy, not on intuition alone.

Required experimental outputs

For every required run, you must save:

- Program output

- gem5 statistics file or extracted statistics
- Exact command used
- You must organize these clearly inside your submission folder.

Recommended commands

The exact Docker invocation may depend on your system setup, but once inside the required container, a typical workflow is:

make

```
/opt/gem5/build/RISCV/gem5.opt run_timing.py --binary ./gemm_accel.riscv --n 64 --mode baseline
```

```
/opt/gem5/build/RISCV/gem5.opt run_timing.py --binary ./gemm_accel.riscv --n 64 --mode accel_medium
```

```
/opt/gem5/build/RISCV/gem5.opt run_minor.py --binary ./gemm_accel.riscv --n 64 --mode baseline
```

```
/opt/gem5/build/RISCV/gem5.opt run_minor.py --binary ./gemm_accel.riscv --n 64 --mode accel_medium
```

Repeat for all required sizes and modes.

Required submission structure

Your ZIP file must contain exactly one top-level folder named:

Project4_<studentID>

Inside it, use the following structure:

Project4_<studentID>/

README.txt

report.pdf

src/

scripts/

results/

results/raw/

results/stats/

results/tables/

plots/

commands.txt

Required report format

Submit one PDF report named report.pdf and having a maximum of 8 pages, excluding references if any.

Required report structure

1. Introduction
2. Baseline CPU characterization
3. Accelerator timing model
4. Offload implementations
5. Experimental methodology
6. Results
7. Granularity and batching analysis
8. Hybrid strategy
9. Energy-oriented analysis
10. Final architecture recommendation

Required results table format

Your report must contain at least one table in the following format:

- N
- CPU model
- mode
- tile size
- offload calls
- simTicks
- simInsts
- checksum
- correct

You may add additional columns if useful.

Required energy table format

Your report must contain at least one table in the following format:

- N
- mode
- CPU MACs
- accelerator MACs
- offload calls
- transfer bytes
- estimated energy

Grading rubric

- Correctness and reproducibility - 30 points
- Accelerator offload implementation quality - 25 points
- gem5 measurement quality - 25 points
- Granularity, batching, and hybrid analysis - 30 points
- Energy-oriented reasoning - 20 points
- Final architecture recommendation - 20 points
- Report quality and organization - 20 points
- **Total** - 170 points, scaled to 10 course marks

Final checklist before submission

- The project builds correctly
- All required modes run
- All required outputs show correct=yes
- All required gem5 runs are completed
- The hybrid strategy is clearly defined
- The energy model is applied correctly
- The report includes all required sections
- The ZIP file has the required structure
- The final recommendation is supported by measured and modeled evidence